

LAPORAN PRAKTIKUM 4
PEMROGRAMAN BERORIENTASI OBJEK
INHERITANCE (PEWARISAN)



Disusun Oleh:

Nama : Mandanta Guru Singa
NIM : 124140147
Mata Kuliah : Pemrograman Berorientasi Objek
Dosen Pengampu : Rahman Indra Kesuma, S. Kom., M. Cs.
Kelas : RB

PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA
2026

1. Tujuan Praktikum

- Menjelaskan konsep inheritance dalam OOP
- Memahami hubungan superclass dan subclass
- Menggunakan keyword extends
- Mengakses atribut dan method dari class induk
- Menggunakan keyword super
- Menerapkan inheritance dalam studi kasus sederhana
- Memahami manfaat penggunaan inheritance

2. Alat dan Bahan

- Laptop
- VS Code
- JDK versi 25
- Modul Praktikum PBO 4

3. Langkah Percobaan

3.1. Percobaan 1

- Membuat class Hewan dengan atribut nama, dan umur disertai method makan() dan tidur().



```
1  class Hewan{
2      String nama;
3      int umur;
4
5      public Hewan(String nama, int umur) {
6          this.nama = nama;
7          this.umur = umur;
8      }
9
10     public void makan() {
11         System.out.println(nama + " sedang makan.");
12     }
13
14     public void tidur() {
15         System.out.println(nama + " sedang tidur.");
16     }
17 }
```

3.2. Percobaan 2

- Membuat class Kucing yang merupakan turunan dari Hewan beserta atribut ras, method mengeong(), method berburu().

```
1 class Kucing extends Hewan {
2     String ras;
3
4     public Kucing(String nama, int umur, String ras) {
5         super(nama, umur);
6         this.ras = ras;
7     }
8     public void mengeong() {
9         System.out.println(nama + " sedang mengeong.");
10    }
11    public void berburu() {
12        System.out.println(nama + " sedang berburu.");
13    }
14 }
```

3.3. Percobaan 3

- Membuat class Anjing yang merupakan turunan dari Hewan beserta atribut jenisGolongan, method menggonggong(), method bermain().

```
1 class Anjing extends Hewan {
2     String jenisGolongan;
3
4     public Anjing(String nama, int umur, String jenisGolongan) {
5         super(nama, umur);
6         this.jenisGolongan = jenisGolongan;
7     }
8     public void menggonggong() {
9         System.out.println(nama + " sedang menggonggong.");
10    }
11    public void bermain() {
12        System.out.println(nama + " sedang bermain.");
13    }
14 }
```

3.4. Percobaan 4

- Mengubah class Hewan, Kucing, dan Anjing agar menggunakan constructor (menggunakan keyword super pada subclass).

```

1 public Hewan(String nama, int umur) {
2     this.nama = nama;
3     this.umur = umur;
4 }
5
6 public Kucing(String nama, int umur, String ras) {
7     super(nama, umur);
8     this.ras = ras;
9 }
10 public Anjing(String nama, int umur, String jenisGolongan) {
11     super(nama, umur);
12     this.jenisGolongan = jenisGolongan;
13 }

```

3.5. Percobaan 5

- Membuat class GoldenRetriever yang merupakan turunan dari Anjing beserta method berenang(), method mengambil().

```

1 class GoldenRetriever extends Anjing {
2     public GoldenRetriever(String nama, int umur) {
3         super(nama, umur, "Golden Retriever");
4     }
5     public void berenang() {
6         System.out.println(nama + " sedang berenang.");
7     }
8     public void mengambil() {
9         System.out.println(nama + " sedang mengambil bola.");
10    }
11 }
12
13 public class Main {
14     public static void main(String[] args) {
15         GoldenRetriever anjl = new GoldenRetriever("Gold Roger", 5);
16         anjl.makan();
17         anjl.tidur();
18         anjl.menggonggong();
19         anjl.bermain();
20         anjl.berenang();
21         anjl.mengambil();
22     }
23 }

```

- Full Code

```
1  class Hewan{
2      String nama;
3      int umur;
4
5      public Hewan(String nama, int umur) {
6          this.nama = nama;
7          this.umur = umur;
8      }
9
10     public void makan() {
11         System.out.println(nama + " sedang makan.");
12     }
13
14     public void tidur() {
15         System.out.println(nama + " sedang tidur.");
16     }
17 }
18
19 class Kucing extends Hewan {
20     String ras;
21
22     public Kucing(String nama, int umur, String ras) {
23         super(nama, umur);
24         this.ras = ras;
25     }
26     public void mengeong() {
27         System.out.println(nama + " sedang mengeong.");
28     }
29     public void berburu() {
30         System.out.println(nama + " sedang berburu.");
31     }
32 }
33
34 class Anjing extends Hewan {
35     String jenisGolongan;
36
37     public Anjing(String nama, int umur, String jenisGolongan) {
38         super(nama, umur);
39         this.jenisGolongan = jenisGolongan;
40     }
41     public void menggonggong() {
42         System.out.println(nama + " sedang menggonggong.");
43     }
44     public void bermain() {
45         System.out.println(nama + " sedang bermain.");
46     }
47 }
48
49 public class Main {
50     public static void main(String[] args) {
51         Hewan kucing = new Hewan("Kucing", 3);
52         Hewan anjing = new Hewan("Anjing", 5);
53
54         kucing.makan();
55         kucing.tidur();
56
57         anjing.makan();
58         anjing.tidur();
59     }
60 }
61
62 public Hewan(String nama, int umur) {
63     this.nama = nama;
64     this.umur = umur;
65 }
66
67 public Kucing(String nama, int umur, String ras) {
68     super(nama, umur);
69     this.ras = ras;
70 }
71 public Anjing(String nama, int umur, String jenisGolongan) {
72     super(nama, umur);
73     this.jenisGolongan = jenisGolongan;
74 }
75
```

4. Source Code

<https://github.com/DanamiStartrail/Tugas-Laporan-PBO/tree/Pertemuan-4>

5. Hasil Output Program

```
Gold Roger sedang makan.  
Gold Roger sedang tidur.  
Gold Roger sedang menggonggong.  
Gold Roger sedang bermain.  
Gold Roger sedang berenang.  
Gold Roger sedang mengambil bola.
```

6. Analisis Program

Program Java ini menerapkan konsep **Inheritance** (Pewarisan) untuk memodelkan hubungan antar berbagai jenis hewan. Struktur kode terdiri dari empat kelas: Hewan, Kucing, Anjing, dan GoldenRetriever, serta satu kelas utama Main.

Program ini juga menerapkan **Multilevel Inheritance** (Pewarisan Bertingkat), di mana kelas GoldenRetriever menjadi subclass dari Anjing. Alurnya adalah: Hewan → Anjing → GoldenRetriever. Hal ini membuat objek GoldenRetriever secara otomatis memiliki akses ke method milik Anjing (seperti menggonggong()) dan juga method dari Hewan secara tidak langsung.

Penggunaan keyword `super` terlihat pada setiap constructor subclass. Misalnya, pada kelas Kucing dan Anjing, `super(nama, umur)` digunakan untuk memanggil constructor superclass (Hewan) agar pengisian atribut umum tidak perlu dilakukan berulang kali. Khusus pada GoldenRetriever, constructor-nya langsung menetapkan nilai "Golden Retriever" ke parameter `jenisGolongan` pada superclass-nya (Anjing).

Pada bagian main, dilakukan instansiasi objek `anj1` dengan tipe GoldenRetriever. Objek ini berhasil memanggil method dari semua level hierarki (dari kelas induk paling umum hingga kelas spesifik miliknya sendiri), yang membuktikan bahwa semakin ke bawah sebuah hierarki, kelas tersebut menjadi semakin spesifik.

7. Pertanyaan Diskusi

7.1. Apa yang dimaksud dengan inheritance dalam OOP?

Inheritance adalah konsep di mana sebuah class dapat mewarisi atribut dan method dari class lain, sehingga kita tidak perlu menulis ulang kode yang sama.

7.2. Apa perbedaan superclass dan subclass?

Superclass (parent class) adalah kelas yang diwarisi, sedangkan subclass (child class) adalah kelas yang menerima warisan atau yang mewarisi.

7.3. Mengapa inheritance dapat mengurangi duplikasi kode?

Karena method dan atribut yang umum cukup ditulis sekali di superclass, dan semua subclass otomatis mendapatkannya tanpa perlu menyalin ulang kode tersebut.

7.4. Apa fungsi keyword super?

Keyword super berfungsi untuk mengakses anggota (seperti constructor atau method) milik superclass secara eksplisit dari dalam subclass.

7.5. Mengapa GoldenRetriever dapat menggunakan method milik Hewan?

Karena GoldenRetriever adalah turunan dari Anjing, dan Anjing adalah turunan dari Hewan. Melalui hubungan bertingkat ini, GoldenRetriever mewarisi apa yang dimiliki Hewan secara tidak langsung.

7.6. Apa yang dimaksud dengan multilevel inheritance?

Multilevel inheritance atau pewarisan bertingkat adalah kondisi ketika sebuah subclass diturunkan lagi menjadi subclass baru, membentuk hierarki lebih dari dua tingkat.

7.7. Kapan inheritance sebaiknya digunakan?

Inheritance sebaiknya digunakan jika hubungan antar class benar-benar menunjukkan hubungan "adalah" (*is-a*)

8. Kesimpulan

Berdasarkan hasil analisis program dan diskusi mengenai konsep **Inheritance** (Pewarisan) pada Modul 4, dapat disimpulkan bahwa:

- **Efisiensi Kode melalui Reusabilitas:** Inheritance memungkinkan pembuatan hierarki kelas yang terstruktur, di mana atribut dan method umum cukup didefinisikan satu kali pada superclass (Hewan). Hal ini secara drastis mengurangi duplikasi kode karena subclass (Kucing dan Anjing) dapat langsung menggunakan fungsi tersebut tanpa penulisan ulang.
- **Hierarki yang Logis dengan Multilevel Inheritance:** Penerapan pewarisan bertingkat pada kelas GoldenRetriever menunjukkan bahwa Java mampu memodelkan hubungan dunia nyata yang kompleks. Objek pada level terbawah hierarki memiliki kemampuan akumulatif, yaitu mewarisi sifat dari nenek moyangnya (Hewan) serta induk langsungnya (Anjing).
- **Kontrol Inisialisasi dengan Keyword super:** Penggunaan super() dalam constructor memastikan bahwa proses inisialisasi data pada superclass tetap

berjalan meskipun kita membuat objek dari subclass. Ini menjamin integritas data (seperti nama dan umur) tetap terjaga dalam struktur pewarisan.

- **Spesialisasi Objek:** Melalui inheritance, kita dapat memperluas fungsi kelas dasar menjadi lebih spesifik. Sementara Hewan hanya bisa makan dan tidur, subclass seperti GoldenRetriever memiliki kemampuan khusus seperti berenang() dan mengambil(), yang membedakannya dari hewan lain namun tetap mempertahankan identitasnya sebagai seekor hewan.
- **Fondasi untuk Polymorphism:** Pemahaman yang matang mengenai bagaimana subclass mewarisi sifat dari superclass menggunakan keyword extends merupakan syarat mutlak sebelum mempelajari konsep *Polymorphism*. Tanpa struktur pewarisan yang benar, implementasi perubahan perilaku objek pada materi berikutnya tidak akan bisa dilakukan.

Secara keseluruhan, program ini telah mendemonstrasikan bagaimana konsep *Inheritance* digunakan untuk menciptakan struktur kode yang ringkas, terorganisir, dan mencerminkan hubungan antar objek secara alami dalam pemrograman berorientasi objek.